

RoboPet — Phase 2, Steps 5 & 6

Room-Based Flame UI: Living Room, Fabricator & Laboratory

Goal. Replace the Phase 1.5 flat PetScreen with a true room-based Tamagotchi experience built on Flame. One FlameGame hosts three swappable World objects. Flutter overlays handle all data-bound UI via Riverpod. The backend (providers, EconomySyncService, GachaService) is untouched.

1. Architecture overview

One RoboPetGame (FlameGame) owns a CameraComponent that points at the active World. Room switch = swap the World + toggle named Flutter overlays. All Riverpod reads happen exclusively in the overlay layer.

Key rule: Flame handles animation, sprites, particles. Flutter overlays consume Riverpod providers. This avoids coupling Flame to WidgetRef.

2. Room enum & RoboPetGame

`lib/game/robot_game.dart`

```
import 'package:flame/game.dart';
import 'package:flame/camera.dart';
import 'rooms/living_room.dart';
import 'rooms/fabricator_room.dart';
import 'rooms/laboratory_room.dart';

enum Room { living, fabricator, laboratory }

const kOverlayHud = 'hud';
const kOverlayNav = 'nav';
const kOverlayFabricator = 'fabricator';
const kOverlayLoadout = 'loadout';

class RoboPetGame extends FlameGame {
  RoboPetGame() : super(camera: CameraComponent());

  late final LivingRoomWorld _living;
  late final FabricatorRoomWorld _fabricator;
  late final LaboratoryRoomWorld _laboratory;

  Room _activeRoom = Room.living;
  Room get activeRoom => _activeRoom;

  @override
  Future<void> onLoad() async {
    await super.onLoad();
    _living = LivingRoomWorld();
  }
}
```

```

    _fabricator = FabricatorRoomWorld();
    _laboratory = LaboratoryRoomWorld();
    world = _living;
    camera.world = _living;
    overlays.add(kOverlayHud);
    overlays.add(kOverlayNav);
  }

  void navigateTo(Room room) {
    if (room == _activeRoom) return;
    _toggleRoomOverlay(_activeRoom, false);
    final next = _worldFor(room);
    world = next;
    camera.world = next;
    _activeRoom = room;
    _toggleRoomOverlay(room, true);
  }

  void _toggleRoomOverlay(Room room, bool show) {
    final key = switch (room) {
      Room.fabricator => kOverlayFabricator,
      Room.laboratory => kOverlayLoadout,
      Room.living => null,
    };
    if (key == null) return;
    show ? overlays.add(key) : overlays.remove(key);
  }

  World _worldFor(Room room) => switch (room) {
    Room.living => _living,
    Room.fabricator => _fabricator,
    Room.laboratory => _laboratory,
  };

  FabricatorRoomWorld get fabricatorWorld => _fabricator;
}

```

3. Living Room (idle wander)

`lib/game/rooms/living_room.dart`

```

import 'dart:math';
import 'package:flame/components.dart';
import 'package:flame/game.dart';

class LivingRoomWorld extends World {
  @override
  Future<void> onLoad() async {
    await super.onLoad();
    add(RectangleComponent(
      size: Vector2(400, 700), position: Vector2(-200, -350),
      paint: Paint()..color = const Color(0xFF0D0F1A)));
    add(RectangleComponent(
      size: Vector2(400, 12), position: Vector2(-200, 120),
      paint: Paint()..color = const Color(0xFF1A2040)));
    add(RobotWanderComponent(position: Vector2(0, 90)));
  }
}

class RobotWanderComponent extends RectangleComponent with HasGameRef {
  RobotWanderComponent({required super.position})
    : super(size: Vector2(48, 64), anchor: Anchor.bottomCenter);
}

```

```

static const _speed = 55.0;
static const _pauseMin = 1.2;
static const _pauseMax = 3.0;
static const _wanderRange = 130.0;

final _rng = Random();
Vector2 _target = Vector2.zero();
double _pauseTimer = 0;
bool _pausing = true;

@override
Paint get paint => Paint()..color = const Color(0xFF22E1FF);

@override
void onMount() {
  super.onMount();
  _target = position.clone();
  _schedulePause();
}

@override
void update(double dt) {
  super.update(dt);
  if (_pausing) {
    _pauseTimer -= dt;
    if (_pauseTimer <= 0) {
      _pausing = false;
      _target = Vector2(
        (_rng.nextDouble() * 2 - 1) * _wanderRange, position.y);
    }
  } else {
    final dir = _target - position;
    if (dir.length < 2) {
      position.setFrom(_target);
      _pausing = true;
      _schedulePause();
    } else {
      position += dir.normalized() * _speed * dt;
    }
  }
}

void _schedulePause() =>
  _pauseTimer = _pauseMin + _rng.nextDouble() * (_pauseMax - _pauseMin);
}

```

4. Fabricator Room (gacha reveal)

lib/game/rooms/fabricator_room.dart

```

import 'dart:math';
import 'package:flame/components.dart';
import 'package:flame/particles.dart';
import 'package:flutter/material.dart' show Color, Paint, Colors;

class FabricatorRoomWorld extends World {
  late final GachaCrateComponent gachaCrate;

  @override
  Future<void> onLoad() async {
    await super.onLoad();
    add(RectangleComponent(
      size: Vector2(400, 700), position: Vector2(-200, -350),

```

```

        paint: Paint()..color = const Color(0xFF060818)));
    add(RectangleComponent(
      size: Vector2(120, 10), position: Vector2(-60, 50),
      paint: Paint()..color = const Color(0xFF334466)));
    gachaCrate = GachaCrateComponent(position: Vector2(0, 20));
    add(gachaCrate);
  }

  void triggerReveal(String rarity) => gachaCrate.reveal(rarity);
}

class GachaCrateComponent extends RectangleComponent with HasGameRef {
  GachaCrateComponent({required super.position})
    : super(size: Vector2(56, 56), anchor: Anchor.center);

  static const _idleAmpY = 8.0;
  static const _idleFreq = 1.4;
  static const _spinDuration = 0.9;

  double _t = 0; double _spinTimer = 0; bool _spinning = false;
  final Vector2 _origin = Vector2.zero();
  static const _colors = {
    'common': Color(0xFFAAAAAA), 'rare': Color(0xFF4488FF), 'epic': Color(0xFFCC44FF) };

  @override
  void onMount() { super.onMount(); _origin.setFrom(position);
    paint.color = const Color(0xFF22E1FF); }

  @override
  void update(double dt) {
    super.update(dt);
    if (_spinning) {
      _spinTimer -= dt; angle += dt * 8.0;
      if (_spinTimer <= 0) { _spinning = false; angle = 0; }
      return;
    }
    _t += dt;
    position.y = _origin.y + _idleAmpY * sin(_t * _idleFreq * 2 * pi);
  }

  void reveal(String rarity) {
    if (_spinning) return;
    _spinning = true; _spinTimer = _spinDuration;
    paint.color = _colors[rarity] ?? _colors['common']!;
    _emitBurst(rarity);
  }

  void _emitBurst(String rarity) {
    final c = _colors[rarity] ?? _colors['common']!;
    parent?.add(ParticleSystemComponent(
      position: position.clone(),
      particle: Particle.generate(
        count: 28, lifespan: 0.9,
        generator: (i) => AcceleratedParticle(
          acceleration: Vector2(
            (Random().nextDouble()*2-1)*60, -120-Random().nextDouble()*80),
          speed: Vector2(
            (Random().nextDouble()*2-1)*90, -60-Random().nextDouble()*60),
          child: CircleParticle(
            radius: 3+Random().nextDouble()*3,
            paint: Paint()..color = c))));
  }
}

```

5. Laboratory Room (loadout)

lib/game/rooms/laboratory_room.dart

```
import 'package:flame/components.dart';
import 'package:flutter/material.dart' show Color, Paint;

class LaboratoryRoomWorld extends World {
  @override
  Future<void> onLoad() async {
    await super.onLoad();
    add(RectangleComponent(
      size: Vector2(400, 700), position: Vector2(-200, -350),
      paint: Paint()..color = const Color(0xFF080D10)));
    add(RectangleComponent(
      size: Vector2(260, 14), position: Vector2(-130, 80),
      paint: Paint()..color = const Color(0xFF1A3040)));
    // Robot display on workbench.
    add(RectangleComponent(
      size: Vector2(48, 64), position: Vector2(-24, 16),
      paint: Paint()..color = const Color(0xFF36F1A6)));
  }
}
```

6. GameScreen (Flutter host)

lib/ui/game_screen.dart

```
import 'package:flame/game.dart';
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../game/robot_game.dart';
import 'overlays/hud_overlay.dart';
import 'overlays/nav_bar_overlay.dart';
import 'overlays/fabricator_overlay.dart';
import 'overlays/loadout_overlay.dart';

class GameScreen extends ConsumerStatefulWidget {
  const GameScreen({super.key});
  @override
  ConsumerState<GameScreen> createState() => _GameScreenState();
}

class _GameScreenState extends ConsumerState<GameScreen> {
  late final RoboPetGame _game;

  @override
  void initState() { super.initState(); _game = RoboPetGame(); }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: Colors.black,
      body: GameWidget<RoboPetGame>(
        game: _game,
        overlayBuilderMap: {
          kOverlayHud: (ctx, game) => HudOverlay(game: game),
          kOverlayNav: (ctx, game) => NavBarOverlay(game: game as RoboPetGame),
          kOverlayFabricator: (ctx, game) =>
            FabricatorOverlay(game: game as RoboPetGame),
          kOverlayLoadout: (ctx, game) =>

```

```

        LoadoutOverlay(game: game as RoboPetGame),
      ),
    ),
  );
}
}

```

7. HUD overlay (currencies)

`lib/ui/overlays/hud_overlay.dart`

```

import 'package:flame/game.dart';
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../../data/providers.dart';
import '../theme.dart';

class HudOverlay extends ConsumerWidget {
  const HudOverlay({super.key, required this.game});
  final FlameGame game;

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final currencies = ref.watch(currenciesStreamProvider);
    return Positioned(
      top: MediaQuery.of(context).padding.top + 8,
      left: 12, right: 12,
      child: Row(
        mainAxisAlignment: MainAxisAlignment.spaceBetween,
        children: [
          _chip(Icons.bolt, currencies.valueOrNull?.scrap ?? 0, AppColors.amber),
          _chip(Icons.settings, currencies.valueOrNull?.rareCogs ?? 0, AppColors.cyan),
          _chip(Icons.diamond, currencies.valueOrNull?.premiumGems ?? 0, AppColors.magenta),
        ]),
    );
  }

  Widget _chip(IconData icon, int value, Color color) => Container(
    padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
    decoration: BoxDecoration(
      color: Colors.black54,
      border: Border.all(color: color.withOpacity(0.6)),
      borderRadius: BorderRadius.circular(20)),
    child: Row(mainAxisSize: MainAxisSize.min, children: [
      Icon(icon, color: color, size: 15),
      const SizedBox(width: 4),
      Text('$value', style: TextStyle(
        color: color, fontSize: 13, fontWeight: FontWeight.bold)),
    ]));
  }
}

```

8. Nav bar overlay (room tabs)

`lib/ui/overlays/nav_bar_overlay.dart`

```

import 'package:flutter/material.dart';
import '../../game/robot_game.dart';
import '../theme.dart';

```

```

class NavBarOverlay extends StatelessWidget {
  const NavBarOverlay({super.key, required this.game});
  final RoboPetGame game;

  @override
  Widget build(BuildContext context) {
    return Positioned(
      bottom: 0, left: 0, right: 0,
      child: Container(
        height: 72 + MediaQuery.of(context).padding.bottom,
        decoration: BoxDecoration(
          color: Colors.black87,
          border: Border(top: BorderSide(color: AppColors.cyan.withOpacity(0.3))),
        ),
        child: SafeArea(
          top: false,
          child: Row(
            mainAxisAlignment: MainAxisAlignment.spaceAround,
            children: [
              _NavTab(icon: Icons.home_filled,
                label: 'Living Room', room: Room.living, game: game),
              _NavTab(icon: Icons.precision_manufacturing,
                label: 'Fabricator', room: Room.fabricator, game: game),
              _NavTab(icon: Icons.science,
                label: 'Laboratory', room: Room.laboratory, game: game),
            ]),
        ),
    );
  }
}

class _NavTab extends StatelessWidget {
  const _NavTab({required this.icon, required this.label,
    required this.room, required this.game});
  final IconData icon; final String label;
  final Room room; final RoboPetGame game;

  @override
  Widget build(BuildContext context) {
    final active = game.activeRoom == room;
    final color = active ? AppColors.cyan : Colors.white38;
    return GestureDetector(
      onTap: () => game.navigateTo(room),
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Icon(icon, color: color, size: 26),
          const SizedBox(height: 3),
          Text(label, style: TextStyle(
            color: color, fontSize: 10, fontWeight: FontWeight.w600)),
        ],
      ),
    );
  }
}

```

9. Fabricator overlay (roll UI + reveal card)

`lib/ui/overlays/fabricator_overlay.dart`

```

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../../data/providers.dart';
import '../../data/gacha/gacha_service.dart';
import '../../game/robot_game.dart';
import '../theme.dart';

```

```

class FabricatorOverlay extends ConsumerStatefulWidget {
  const FabricatorOverlay({super.key, required this.game});
  final RoboPetGame game;
  @override
  ConsumerState<FabricatorOverlay> createState() => _FabricatorOverlayState();
}

class _FabricatorOverlayState extends ConsumerState<FabricatorOverlay> {
  bool _rolling = false;
  String? _lastRarity; String? _lastDefId; String? _error;

  Future<void> _roll() async {
    setState(() { _rolling = true; _error = null; });
    try {
      final result = await ref.read(gachaServiceProvider).roll();
      setState(() { _lastRarity = result.rarity.name; _lastDefId = result.defId; });
      widget.game.fabricatorWorld.triggerReveal(result.rarity.name);
    } on GachaException catch (e) {
      setState(() => _error = e.message);
    } finally {
      setState(() => _rolling = false);
    }
  }

  @override
  Widget build(BuildContext context) {
    final cogs = ref.watch(currenciesStreamProvider).valueOrNull?.rareCogs ?? 0;
    return Positioned(
      bottom: 88, left: 20, right: 20,
      child: Column(mainAxisSize: MainAxisSize.min, children: [
        if (_lastRarity != null)
          _RevealCard(rarity: _lastRarity!, defId: _lastDefId!),
        if (_error != null)
          Padding(padding: const EdgeInsets.only(bottom: 8),
            child: Text(_error!, style:
              const TextStyle(color: Colors.redAccent, fontSize: 13))),
        _GlowButton(
          label: _rolling ? 'Fabricating...' : 'Fabricate ⚙️ 1 RareCog',
          enabled: cogs >= 1 && !_rolling,
          color: AppColors.magenta, onTap: _roll),
        Text('$cogs RareCogs available',
          style: TextStyle(color: AppColors.cyan, fontSize: 11)),
      ]));
  }
}

class _RevealCard extends StatelessWidget {
  const _RevealCard({required this.rarity, required this.defId});
  final String rarity; final String defId;
  static const _colors = {
    'common': Color(0xFFAAAAAA), 'rare': Color(0xFF4488FF), 'epic': Color(0xFFCC44FF) };
  @override
  Widget build(BuildContext context) {
    final c = _colors[rarity] ?? _colors['common']!;
    return Container(
      margin: const EdgeInsets.only(bottom: 12),
      padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 12),
      decoration: BoxDecoration(
        color: Colors.black87, border: Border.all(color: c, width: 1.5),
        borderRadius: BorderRadius.circular(12),
        boxShadow: [BoxShadow(color: c.withOpacity(0.3), blurRadius: 16)]),
      child: Row(mainAxisAlignment: MainAxisAlignment.center, children: [
        Icon(Icons.memory, color: c, size: 22),
        const SizedBox(width: 10),
        Text(defId.replaceAll('_', ' ').toUpperCase(),
          style: TextStyle(color: c, fontWeight: FontWeight.bold,
            fontSize: 13, letterSpacing: 1.2)),
        const SizedBox(width: 10),
        Text(rarity.toUpperCase(), style: TextStyle(

```



```

        color: c.withOpacity(0.7), fontSize: 10, letterSpacing: 1.5)),
    ]));
  }
}

class _GlowButton extends StatelessWidget {
  const _GlowButton({required this.label, required this.enabled,
    required this.color, required this.onTap});
  final String label; final bool enabled;
  final Color color; final VoidCallback onTap;
  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      onTap: enabled ? onTap : null,
      child: AnimatedContainer(
        duration: const Duration(milliseconds: 200),
        margin: const EdgeInsets.only(bottom: 8),
        padding: const EdgeInsets.symmetric(vertical: 14),
        alignment: Alignment.center,
        decoration: BoxDecoration(
          color: enabled ? color.withOpacity(0.15) : Colors.white10,
          border: Border.all(color: enabled ? color : Colors.white24),
          borderRadius: BorderRadius.circular(10),
          boxShadow: enabled ? [BoxShadow(color: color.withOpacity(0.35), blurRadius: 18)] :
[]),
        child: Text(label, style: TextStyle(
          color: enabled ? color : Colors.white30,
          fontWeight: FontWeight.bold, fontSize: 14))));
  }
}

```

10. Loadout overlay (slot grid + module picker)

lib/ui/overlays/loadout_overlay.dart

```

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../data/providers.dart';
import '../data/models/module_enums.dart';
import '../data/models/module_instance.dart';
import '../game/robot_game.dart';
import '../theme.dart';

class LoadoutOverlay extends ConsumerWidget {
  const LoadoutOverlay({super.key, required this.game});
  final RoboPetGame game;

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final inventory = ref.watch(inventoryProvider);
    const robotId = 'robot_0'; // swap with ref.watch(activeRobotIdProvider)
    final equipped = ref.watch(equippedModulesProvider(robotId));
    return Positioned(
      bottom: 88, left: 0, right: 0,
      child: Column(mainAxisSize: MainAxisSize.min, children: [
        Padding(
          padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 4),
          child: Text('LOADOUT', style: TextStyle(
            color: AppColors.green, fontSize: 11,
            letterSpacing: 2, fontWeight: FontWeight.bold))),
        SizedBox(
          height: 74,
          child: ListView(

```

```

        scrollDirection: Axis.horizontal,
        padding: const EdgeInsets.symmetric(horizontal: 12),
        children: ModuleSlot.values.map((slot) => _SlotCard(
            slot: slot, equipped: equipped[slot],
            onTap: () => _openPicker(context, ref, slot, robotId,
                inventory.valueOrNull ?? []),
            )),
    ));
}

void _openPicker(BuildContext ctx, WidgetRef ref, ModuleSlot slot,
    String robotId, List<ModuleInstance> inventory) {
    showModalBottomSheet(
        context: ctx,
        backgroundColor: const Color(0xFF0D1220),
        shape: const RoundedRectangleBorder(
            borderRadius: BorderRadius.vertical(top: Radius.circular(16))),
        builder: (_) => _ModulePicker(
            slot: slot,
            modules: inventory.where((m) => m.def?.slot == slot).toList(),
            robotId: robotId));
}

class _SlotCard extends StatelessWidget {
    const _SlotCard({required this.slot, required this.equipped, required this.onTap});
    final ModuleSlot slot; final ModuleInstance? equipped; final VoidCallback onTap;
    static const _icons = {
        ModuleSlot.wheels: Icons.rotate_right, ModuleSlot.armor: Icons.shield,
        ModuleSlot.sensor: Icons.sensors, ModuleSlot.core: Icons.memory,
        ModuleSlot.utility: Icons.electrical_services };
    static const _rc = {
        'common': Color(0xFFAAAAAA), 'rare': Color(0xFF4488FF), 'epic': Color(0xFFCC44FF) };
    @override
    Widget build(BuildContext context) {
        final c = equipped != null ? (_rc[equipped!.rarity.name] ?? Colors.white30) :
Colors.white24;
        return GestureDetector(
            onTap: onTap,
            child: Container(
                width: 64, height: 64, margin: const EdgeInsets.only(right: 10),
                decoration: BoxDecoration(
                    color: Colors.black54, border: Border.all(color: c),
                    borderRadius: BorderRadius.circular(10),
                    boxShadow: equipped != null ? [BoxShadow(color: c.withOpacity(0.3), blurRadius:
10)] : []),
                child: Column(mainAxisAlignment: MainAxisAlignment.center, children: [
                    Icon(_icons[slot], color: c, size: 22),
                    const SizedBox(height: 2),
                    Text(slot.name, style: TextStyle(color: c, fontSize: 9, letterSpacing: 0.5)),
                ])));
    }
}

class _ModulePicker extends ConsumerWidget {
    const _ModulePicker({required this.slot, required this.modules, required this.robotId});
    final ModuleSlot slot; final List<ModuleInstance> modules; final String robotId;
    @override
    Widget build(BuildContext context, WidgetRef ref) {
        if (modules.isEmpty) {
            return Padding(padding: const EdgeInsets.all(32),
                child: Center(child: Text('No ${slot.name} modules in storage.',
                    style: const TextStyle(color: Colors.white54, fontSize: 14))));
        }
        return ListView.builder(
            padding: const EdgeInsets.all(16), itemCount: modules.length,
            itemBuilder: (_, i) {
                final m = modules[i];
                return ListTile(

```

```

        leading: Icon(Icons.memory,
            color: m.equippedRobotId != null ? Colors.white30 : AppColors.cyan),
        title: Text(m.def?.name ?? m.defId,
            style: const TextStyle(color: Colors.white, fontSize: 13)),
        subtitle: Text(m.rarity.name.toUpperCase(),
            style: const TextStyle(color: Colors.white38, fontSize: 10)),
        trailing: m.equippedRobotId == robotId
            ? const Text('EQUIPPED', style: TextStyle(color: Colors.white30, fontSize: 10))
            : null,
        onTap: () {
            ref.read(isarProvider).writeTxnSync(() {
                m.equippedRobotId = robotId;
                ref.read(isarProvider).moduleInstances.putSync(m);
            });
            Navigator.pop(context);
        });
    });
}
}

```

11. Verification checklist

- [] GameScreen replaces PetScreen in MaterialApp; app hot-restarts cleanly.
- [] Living Room: robot wanders left/right; HUD shows live currencies from Riverpod.
- [] Fabricator Room: nav tab switches Flame world; Fabricate button calls rollGacha; crate spins and emits particles; reveal card shows rarity-correct border glow.
- [] Insufficient rareCogs: Fabricate button is dimmed; GachaException message appears above button.
- [] Laboratory Room: five slot cards render; tapping an empty slot opens the module picker; equipping a module updates the card border to rarity color instantly.
- [] Nav bar active tab highlights cyan; switching rooms does not reset Riverpod state.

Phase 2 UI complete. Next: Phase 3 -- Combat. resolveStats runs over equipped modules -> CombatSnapshot -> auto-battler simulation. The Laboratory Room can host a Deploy to Battle button. Say the word!